Republic of the Philippines
PAMANTASAN NG LUNGSOD NG MAYNILA
(University of the City of Manila)
General Luna Street corner Muralla Street
Intramuros, Manila

Pothos Compiler

BSCS 3-2 (Group 3)

A.Y 2026-2027

A Compiler presented to the

Computer Science Department

College of Information Systems and Technology Management

In Partial Fulfillment of the Requirements for Degree

Bachelor of Science in Computer Science (BSCS)

Under the Supervision of

Prof. Leisyl C. Mahusay

Submitted by:

Avila, Julien Jamile P.

Bisnan, Leinard Allen B.

Blaza, Charles Aldrin M.

Fernandez, Patrick U.

Francisco, Lj S.



TABLE OF CONTENTS

I. OVERVIEW OF THE LANGUAGE....4
II. STRUCTURE OF THE LANGUAGE....5
III. GENERAL RULES....7
IV. LIST OF RESERVED WORDS....11
V. LIST OF RESERVED SYMBOLS....14
VI. SPECIFIC RULES....16
A. Data Types and Literals....16
A.2 Wharfa....17
A.3 Flo....18
A.4 Trua....19
A.5 Lis....20
A.5.1 Declaration....20
A.5.2 Initialization....21
A.5.3 Indexing....22
A.5.4 Nested lis....23
A.6 Glossare....24
B. Operators....26
C. Identifiers....31
D. Expressions....32
E. Statements and Code Blocks....33
F. Comments....34
G. Variables and Constants....35
H. Assignment and Augmented Assignment....36
I. Built-In Functions....37
I.1 prin()....37
I.2 inpub()....38
I.5 formo()....39
I.6 Display Forms....40
J. Conditional Statements....41
K. Looping Statements....43
K.1 Cycla Loop....43
K.2 Lood Loop....44
L. Loop Control Statements....46
M. Functions....47
M.1 Function Declaration....47
M.2 Parameters....49



M.3 Return Values..... 49
M.4 Function Calls..... 49
M.5 Recursion.....50
M.6 Scope and Visibility.....50
N. Object Removal.....51
VII. REGULAR DEFINITIONS..... 52
VIII. REGULAR EXPRESSION.....55
A. Reserved Words..... 55
B. Reserved symbols..... 56
C. Reserved Literals..... 58
IX. DELIMITERS.....59
X. TRANSITION DIAGRAM..... 63
A. Reserved Words..... 63
B. Reserved Symbols.....65
C. Identifiers..... 67
D. Integer and Float Literals.....68
E. String Literals and Comments.....69

I. OVERVIEW OF THE LANGUAGE



Pothos is a compact, instructional programming language developed to introduce learners to the essential mechanics of compiler construction. Its name draws from the ancient Greek word for longing or yearning, a fitting root for a language whose central goal is to cultivate a deeper understanding of how programming languages work beneath the surface, rather than simply how to use them. Pothos is built on the syntactic structure of C/C++ and Python, using semicolon-terminated statements, curly-brace-delimited blocks, and explicit function return types. For readability, the reserved-word table also maps Pothos keywords to familiar Python counterparts. Unlike traditional languages that require a dedicated main function, Pothos utilizes a script-like execution model where the source file itself serves as the entry point, executing top-level statements sequentially. This hybrid identity defines Pothos: syntax simple enough to read at a glance, yet explicit enough that every rule of the grammar is visible and intentional.

What further sets Pothos apart is its own distinct character naming. Pothos uses thematic, Greek-inspired reserved words. Some keywords are invented or stylized to support the language's identity while still teaching foundational programming concepts. Keywords like wharfa for integers, strina for strings, belial for conditionals, and reor for returns give the language a unique linguistic personality. This design choice gives Pothos a creative identity while still teaching foundational concepts. Furthermore, Pothos embraces automatic type inference for variables and lists, allowing learners to focus on logic rather than verbose type declarations, while still supporting explicit return types for functions. Unique syntactic elements, such as single-line (--) and multi-line (/-- ... --/) comments, alongside built-in I/O functions like prin() and inpub(), expose distinct stages of how a compiler reads, checks, and executes code.



II. STRUCTURE OF THE LANGUAGE

Main file (main.pths) | compile using pothos.py main.pths, run using main.exe

```
/-- Declarations --/

<identifier> = <value>;

<identifier> = [<value>, <value>, ...];

<identifier> = {<strina_key>: <value>, ...};


/-- Optional boolean declaration --/

trua <identifier>;


/-- Function Declaration --/

<return_type> <identifier>(<type> <parameter>, <type> <parameter>, ...){

   <statement/s>;

   reor <expression>;

};
```

```
msg = "Hello, World!";
prin(msg);

wharfa add(wharfa a, wharfa b){
   reor a + b;
};

prin(add(3, 4));
```

In the Pothos language, every program follows a structured, script-like format to maintain consistency and readability. Unlike traditional languages that require a dedicated main function, the source file itself (.pths) serves as the entry point, executing top-level statements sequentially from top to bottom. Single-line comments are indicated by --, and multi-line comments are



enclosed between /-- and --/. Comments describe the code but are ignored during execution. Because Pothos utilizes an automatic type of inference, global and local variables are simply declared by pairing an identifier with a value using an assignment operator (e.g., msg = "Hello";), while lists are initialized using square brackets. Functions are defined by stating an explicit return type (such as wharfa or strina), followed by a unique name and optional parameters enclosed in parentheses. The function body is enclosed in curly brackets {}, where local declarations and executable statements are written, each terminated by a semicolon, and the keyword reor returns a value and exits the function. Because blocks are delimited by curly brackets instead of indentation, and every statement is explicitly terminated, this structure keeps .pths programs organized, readable, and easy to maintain .



III. GENERAL RULES

Pothos is a compiled general-purpose programming language that supports datatypes, console input/output (IO), arithmetic operations, etc. Rules are listed below in topical arrangement.

1. The source code is a file written in plain ASCII text, the file uses the **.pths** extension.
    a. The entry point of a program is the file itself; statements are executed in order from top to bottom.
    b. The Pothos character set is ASCII. Non-ASCII characters (e.g., Ω, ω, emoji, accented letters) are lexical errors wherever they appear.
    c. To run the program, you must compile it first using the pothos compiler. Then run the **exe** program generated.
        i. **pothos.exe helloworld.pths**
        ii. **helloworld.exe**
    d. Usable variables and functions must be declared first to be used.
    e. Functions can access variables defined anywhere, provided the variable has been declared by the time the function is called.
2. Identifiers are case-sensitive names of variables, constants, functions, and parameters.
    a. Global variable names, constant names, and function names are unique program-wide.
    b. Function parameter names are local to the function in which they are declared
    c. Identifier names cannot be named after existing reserved keywords.
    d. Identifier names have a maximum length of 20 characters, with a minimum of one character for variables and three characters for functions.
    e. Identifier names must start with a letter and contain only alphanumeric characters and underscores.
3. A code block is a logical grouping of executable statements delimited by curly braces { }.
    a. Code blocks will execute exclusively under three conditions:
        i. when activated by a conditional statement,
        ii. when activated by a looping statement, or
        iii. when invoked through a function call.



4. Any identifier that links to a value is called a variable.
    a. Variable types are automatically inferred from their assigned values at declaration and are immutable; they cannot be recast.
        i. Value errors immediately crash the program.
    b. A variable is declared by pairing an identifier with a value using an assignment operator, except that a boolean may be declared with **trua** <identifier>;, in which case it defaults to **Inaccu**.
        i. Anything that can return a value can be placed instead of an actual value as long as:
            1. That anything guarantees to return a value.
            2. The value returned matches the variable type.
5. Any identifier that links to a reusable block of code is called a function.
    a. Function definitions are initiated with a return data type, which is immediately followed by the unique name and a comma-separated list of parameters enclosed in parentheses, while the executable block itself must be delimited by curly braces { }.
    b. To invoke a specific function, its unique name must be stated along with any required arguments separated by commas and contained within parentheses ().
    c. Function parameters are not required but must be separated by commas (**,**) if there exist multiple.
    d. To exit the function, it must return a value using the keyword '**reor**'.
    e. The built-in functions are always available and their names are reserved: **prin()**, **inpub**(), **formo**(), **innum**(), **inflo**().
6. Reserved words are written only in lowercase letters, except the truth literals **Trua** and **Inaccu**, which are capitalized and case-sensitive.
7. Comments are ignored by the compiler.
    a. A single line comment is indicated by -- and ends with a newline character.
    b. A multi-line comment is indicated by /-- at the start and ends with --/.
8. A statement is a group of reserve words, literals, values, operators and functions working together.
9. Supported data types are **strina**, **wharfa**, **flo**, **trua**, **lis**, and **glossare**.



a. A **wharfa** literal is an unsigned sequence of digits capped at 8 digits maximum.
    i. A negative integer is formed by applying the unary minus operator to a **wharfa** literal. The lexer does not include - inside **wharfa_lit**.
    ii. Unnecessary leading zeros are discarded or ignored
        1. 0 → 0
        2. 0100 → 100
        3. 00 → 0

b. A **flo** literal is an unsigned decimal number with the integer part capped at 8 significant digits and the fractional part capped at 4 digits.
    i. A negative **flo** is formed by applying the unary minus operator to a **flo** literal. The lexer does not include - inside **flo_lit**.
    ii. Unnecessary leading zeros in the integer part are discarded or ignored.
        1. 0.0000 → 0.0
        2. 0100.50 → 100.5

c. Strings encompass any ASCII character, though embedding the enclosing delimiter and the escape characters requires an escape sequence such as: " → \", \ → \\
    i. Strings also have special escape sequences denoting newline \n, null character \0, \" double quote, \' single quote, \\ backslash, \t horizontal tab.

d. Booleans are variables that can only have two states.
    i. Declaring it blank will automatically set it to **Inaccu** (false) by default.

10. An array or its equivalent, **lis**, is declared by assigning a list of elements enclosed in square brackets([]) to a variable name. Its element type is inferred from its contents.
    a. A **lis** has a fixed length after creation.
    b. Array elements are accessed with square brackets using a zero-based index.
11. The assignment operator is =. The comparison operators are ==, !=, <, <=, >, >=.
12. Arithmetic operators are symbols that perform changes within variables and values. The arithmetic operators included are **+, -, *, /,** and %. The symbols represent the operation accordingly.
13. A conditional statement is written as '**belial**' (condition) { }, with optional additional '**belsa**' (condition) { } branches and an optional final '**elsa**' { } branch.
14. Logical operators evaluate relations between two boolean values using **an** (and), **othan** (or), and **neve** (not).
15. A '**cycla**' (condition) { } loop repeats while the condition is '**Trua**'.



   a. A sequence-form **lood**(<name> **insid** <**lis**, **strina**, or **glossare**>){} loop iterates over each element.
   b. A counted **lood**(<assignment>; <condition>; <assignment>){} loop runs the init once, repeats while the condition is **Trua**, and runs the step after each iteration.
   c. '**paus**'; exits the nearest loop.
   d. '**persis**'; skips to the next iteration.
16. The standalone statement '**pas**'; does nothing.
17. The keyword '**insid**' has two roles: as an operator, it tests whether a value is contained in a **lis** or **strina**.
   a. In a sequence-form **lood** header, '**insid**' introduces the **iterable** (**lis**, **strina**, or **glossare**) whose elements the loop visits; the counted form does not use '**insid**'.



IV. LIST OF RESERVED WORDS

COUNTERPART (C / C++/ Python)	FUNCTIONS/DESCRIPTION	List of Reserved Words (Proposed)	REGULAR EXPRESSION	EXAMPLES
1. **Built-in Data Types**				
str	Represents text strings.	strina	(s)(t)(r)(i)(n)(a)	msg = "Hello"; (implicit declaration) strina printx() {}; (return type)
int	Represents integer numbers.	wharfa	(w)(h)(a)(r)(f)(a)	count = 10; (implicit declaration) wharfa numx() {} (return type)
float	Represents numbers with decimals.	flo	(f)(l)(o)	price = 19.99; (implicit declaration) flo flox() {} (return type)
bool	Represents boolean truth values.	trua	(t)(r)(u)(a)	trua active;
list	Represents ordered sequences.	lis	(l)(i)(s)	nums = [1, 2, 3]; lis items() {} (return type)
dict	Represents key-value mappings.	glossare	(g)(l)(o)(s)(s)(a)(r)(e)	data = {"a": 1}; glossare book() {} (return type)
2. **Input / Output Functions**				



print()	Outputs text to the console.	prin()	(p)(r)(i)(n)	prin("Hello");
input()	Reads input from the user.	inpub()	(i)(n)(p)(u)(b)	txt = inpub();
format()	Formats string values.	formo()	(f)(o)(r)(m)(o)	s = formo("Sum: {x}");
int(input())	Reads one line of input as a wharfa.	innum()	(i)(n)(n)(u)(m)	n = innum();
float(input())	Reads one line of input as a flo.	inflo()	(i)(n)(f)(l)(o)	f = inflo();
3. **Value & Truth Literals**				
False	Boolean false value.	Inaccu	(I)(n)(a)(c)(c)(u)	flag = Inaccu;
True	Boolean true value.	Trua	(T)(r)(u)(a)	flag = Trua;
4. **Logical Operators**				
and	Logical conjunction.	an	(a)(n)	a an b
or	Logical disjunction.	othan	(o)(t)(h)(a)(n)	a othan b
not	Logical negation.	neve	(n)(e)(v)(e)	neve active
5. **Structural Control Flow**				
if	Conditional execution starts.	belial	(b)(e)(l)(i)(a)(l)	belial (x > 0) {}
elif	Conditional 'else if' branch.	belsa	(b)(e)(l)(s)(a)	belsa (x == 0) {}
else	Conditional fallback branch.	elsa	(e)(l)(s)(a)	elsa {}
for	Iterates over a sequence.	lood	(l)(o)(o)(d)	lood (x insid nums) {} (iterator type) lood (x = 0; x < 10; x += 1) {} (legacy type)



while	Loops until condition false.	cycla	(c)(y)(c)(l)(a)	cycla (x > 0) {}
break	Exits a loop.	paus	(p)(a)(u)(s)	paus;
continue	Skips to next iteration.	persis	(p)(e)(r)(s)(i)(s)	persis;
pass	No-op statement	pas	(p)(a)(s)	pas;
6. **Structural Definitions**				
return	Returns value from function.	reor	(r)(e)(o)(r)	reor val;
7. **Variable Operations**				
del	Removes objects/references .	deleo	(d)(e)(l)(e)(o)	deleo temp;
const	Declares a constant	const	(c)(o)(n)(s)(t)	const MAX = 100;
8. **Object & Collection Membership**				
in	Checks sequence membership.	insid	(i)(n)(s)(i)(d)	x insid nums



V. LIST OF RESERVED SYMBOLS

List of Reserved Symbols (Existing)	Functions/Descriptions	List of Symbols Words (Proposed)	Examples
Arithmetic Operators			
+	Addition or string concatenation	+	x = 5 + 3; or s = "A" + "B";
-	Subtraction; unary minus handled by parser	-	x = 5 - 3; or y = -5;
*	Multiplication	*	area = w * h;
/	Integer Division	/	q = 10 / 2;
%	Modulo	%	r = 10 % 3;
Relational Operators			
<	Less-than comparison	<	belial(x < 10){ }
>	Greater-than comparison	>	belial(y > 5){ }
<=	Less-than-or-equal comparison	<=	belial(z <= 100){ }
>=	Greater-than-or-equal comparison	>=	belial(age >= 18){ }
==	Equality comparison	==	belial(name == "Ana"){}
!=	Inequality comparison	!=	belial(x != 0){}
Assignment and Augmented Operators			
=	Assignment	=	score = 100;
+=	Augmented add-assign	+=	count += 1;
-=	Augmented subtract-assign	-=	lives -= 1;
*=	Augmented multiply-assign	*=	total *= 2;
/=	Augmented divide-assign	/=	avg /= n;



%=	Augmented modulo-assign	%=	rem %= 5;
Grouping Symbols			
(	Open parenthesis -- grouping or function call	(	x = (1 + 2) * 3;
)	Close parenthesis -- grouping or function call	)	prin("Hi");
[	Open bracket -- list literal or subscript	[	nums = [1, 2];
]	Close bracket -- list literal or subscript	]	x = nums[0];
{	Open brace -- glossare literal or code block	{	belial(x==1){
}	Close brace -- dict or set literal	}	data = {"a":1};
Others			
,	Separator for arguments or elements	,	nums = [1, 2, 3];
"	Double quote: opens or closes a strina literal	"	msg = "Hello";
'	Single quote: opens or closes a strina literal	'	msg = 'Hello';
:	Key separator in glossare literals	:	data = {"name": "Ana"};
;	Statement separator	;	x = 5;
--	Comment -- rest of line is ignored	--	x = 5; -- this is a comment
/--	Multi-line comment start	/--	/-- start of comment
--/	Multi-line comment end (a --/ with no opening /-- is an error)	--/	end of comment --/



VI. SPECIFIC RULES

A. Data Types and Literals

Pothos has six built-in data types: **strina** (text), **wharfa** (integer), **flo** (decimal), **trua** (boolean), **lis** (list), and **glossare** (dictionary). Literals are fixed values written in the source code.

A.1 Strina

strina is the text type of Pothos, used for sequences of ASCII characters:

1. A **strina** literal is enclosed in a matching pair of double quotes (") or single quotes (').
    a. Mismatched opening and closing delimiters are an error.
    b. Any unclosed text sequence is a lexical error; the tokenizer absorbs the full line following the opening delimiter as an unclosed **strina** token.
2. A **strina** literal may also be empty: "" or ''.
3. Strings encompass any ASCII character, though embedding specific reserved symbols requires appropriate processing.
    a. Embedding the enclosing delimiter and escape characters requires an explicit escape sequence using '\'.
        i. \n denotes newline
        ii. \\ denotes backslash
        iii. \' denotes single quote
        iv. \" denotes double quote
        v. \t denotes horizontal tab
        vi. \0 denotes null character
    b. Enclosing string literals in single quotes permits unescaped double quotes internally, and vice versa.
    c. Non-ASCII characters missing from this specification trigger lexical errors.
4. The enclosing quotation mark must be escaped to appear inside the literal (\" or \').
    a. An unescaped enclosing quote inside the literal is an error.
5. An unescaped newline cannot appear inside a **strina** literal; the escape \n must be used.
6. Two **strina** values may be joined with the + operator.



Syntax
"<characters>" or '<characters>'

Valid Examples	Invalid Examples
title = "Pothos";	bad1 = "Hello; (unterminated string)
double_empty = ""; single_empty = '';	bad2 = 'AB"; (mismatched delimiters)
message = "Line one\nLine two";	bad3 = "Newline here"; (unescaped newline)
quote = "She said,\"Hi.\"";	bad4 = "Hi\x"; (invalid escape sequence)
pothos = "Compiler 'pothos' ";	comment = "Compiler "pothos" comment" ; (missing escape characters)
comment = ' Compiler "pothos" ' ;	comment = 'Compiler 'pothos' comment' ; (missing escape characters)
greet = "hello world";	greet = "hello Ω"; (Non-ASCII characters)

A.2 Wharfa

wharfa is the integer type of Pothos, written without decimal points or fractions:

1. A **wharfa** literal is an unsigned sequence of digits from 0 to 9.
2. A negative **wharfa value** is produced by the unary minus (**-**) operator before the digits; positive literals are written without a plus sign.
    a. A leading '+' is an error.
3. Leading zeros are discarded and do not count toward the digit limit (0100 -> 100).
4. The maximum size is 8 significant digits; the allowed range is -99999999 to 99999999.
    a. Any literal exceeding this range is a syntax error.
        i. In the syntax analyzer it will appear as two **wharfa_lit** next to each other.
    b. The tokenizer absorbs the initial 8 numerical characters and processes any surplus sequence as distinct tokens.
5. Decimal points, commas, underscores, spaces, and exponent symbols are not allowed inside a **wharfa** literal.
    a. ., _, and exponent letters after digits are lexical errors.
    b. Commas and spaces are parser errors.



6. A **wharfa** value may only be assigned to a **wharfa** variable or used where an integer is expected.
    a. Using a **wharfa** where another type is expected is an error.
    b. The exception is that a **wharfa** may operate alongside a **flo** under promotion; the result is a **flo** value.

Valid Examples	**Invalid Examples**
a = 0; b = 100;	x = 100000000; (exceeds 8 significant digits)
c = -99999999;	y = 1,000; (comma not allowed)
d = 0100;	z = +25; (plus sign not allowed)
e = 50;	w = 1.5; (decimal point not allowed)

A.3 Flo

flo is the decimal number type of Pothos, used for numeric values with a fractional part:

1. A **flo** literal is an unsigned sequence of digits, a single decimal point (.), then a second sequence of digits; at least one digit is required on both sides of the point.
    a. A decimal point missing a digit on either side is a lexical error.
2. A negative **flo** value is produced by the unary minus (-) operator before the digits; positive literals are written without a plus sign.
    a. A leading '+' is an error.
3. Leading zeros in the integer part are discarded and do not count toward the digit limit.
4. The integer part is capped at 8 significant digits and the fractional part at 4 digits; the allowed range is -99999999.9999 to 99999999.9999.
    a. Any literal exceeding these caps is a syntax error.
    b. The tokenizer absorbs the maximum allowed digits (8 integer, 4 fractional) and processes any surplus sequence as distinct tokens.
5. Exponent symbols, commas, underscores, and spaces are not allowed inside a **flo** literal.
    a. '_' and exponent letters after digits are lexical errors; a '.' that does not complete a **flo literal** is a lexical error.
    b. Commas and spaces are parser errors.
6. A **flo** value is stored as a fixed-point decimal with exactly 4 fractional digits; arithmetic results are rounded half-up to 4 fractional digits.



   a. Equality (==, !=) and ordering (<, <=, >, >=) compare the stored fixed-point values and produce a **trua** value.
7. When printed by **prin**() or interpolated by **formo**(), a **flo** displays up to 4 fractional digits, trimming trailing zeros but always keeping at least one fractional digit.
8. A **flo** value may only be assigned to a **flo** variable or used where a **flo** is expected.
   a. Using a **flo** where another type is expected is an error.
   b. Mixing **wharfa** and **flo** operands is permitted only inside operator evaluation under the promotion rule.
      i. No implicit conversion exists anywhere else; assignment, augmented assignment, argument passing, and return values remain exact-type.

Syntax
<digits>.<digits>

Valid Examples	**Invalid Examples**
a = 0.0; b = 19.99;	x = 5.; (no digit after the point)
b = 0.5;	y = .5; (no digit before the point)
c = -99999999.9999;	z = +2.5; (plus sign not allowed)
e = 7.0 / 2.0; (→ 3.5)	w = 1.23456; (fractional cap exceeded)
price = 19.99; prin(price);	v = 123456789.1; (integer cap exceeded)
t = 1.0 + 1.5;	u = 1.5e2; (exponent symbol not allowed)
result = 1.0 + 1.5;	s = 5.0 % 2.0; (modulo not defined for flo)
quotient = 5.0 / 2.0;	
t = 1 + 1.5;	

A.4 Trua

trua is the boolean type of Pothos, holding exactly one of two truth states:

1. The type keyword is lowercase: **trua**.
2. The truth literals are capitalized and case-sensitive: **Trua** (true) and **Inaccu** (false).



   a. Any other casing (e.g, true, TRUA) is an error.
3. Numeric substitutes such as 1 or 0 are not valid boolean literals.
4. A **trua** variable may be explicitly declared without initialization (**trua** <identifier>;), in which case it defaults to **Inaccu**.
5. Logical expressions and relational expressions produce **trua** values.

Syntax
trua <identifier>;

Valid Examples	**Invalid Examples**
active = Trua; inactive = Inaccu;	active = true; (invalid literal)
trua flag;	locked = 0; (numeric substitute)
result = 3 > 2;	flag = Trua (missing semicolon)

A.5 Lis

lis is the fixed-length, ordered sequence type of Pothos:

A.5.1 Declaration

1. A **lis** variable is declared by assigning a **lis** literal or **lis-yielding expression** to an identifier; **lis** has no type-keyword declaration form.
   a. **lis** nums; is an error; only **trua** has a blank declaration.
   b. Using a **lis** identifier before its declaration is an error.
2. The element type is inferred from the elements at declaration and is part of the variable's immutable type.
   a. Reassignment must supply a **lis** of the same element type otherwise it will lead to an error.
   b. The new value's length may differ; length belongs to the value, not the variable's type.
3. **Lis-yielding expressions** are: another **lis** variable, or a call to a function whose return type is **lis**.
4. A function may declare **lis** parameters and a **lis** return type; a parameter needs no literal in the header. It is initialized by the argument at call time.
5. The empty **lis** [] carries no element type; it is vacuously homogeneous and accepted anywhere a **lis** is expected.



a. A variable declared [] fixes its element type at the first non-empty **lis** assigned to it; the variable's type remains **lis** throughout, so immutability is not violated.
b. Indexing an **empty-lis-declared** variable before that point is always an out-of-range error; **insid** on it always produces **Inaccu**.

Syntax
<identifier> = [<value>, ...];
<identifier> = <lis variable | lis-returning call>;

Valid Examples	**Invalid Examples**
nums=[1,2,3];	lis nums;; (illegal declaration)
copy=nums;	prin(nums[0]); (undeclared use)
words=labels();	nums=[1,2]; nums=["a","b"]; (type mismatch)
empty=[];	

A.5.2 Initialization

1. A **lis** literal is enclosed in square brackets [], elements separated by commas; it may be empty: [].
    a. A missing closing bracket, missing separator, or trailing comma is an error.
2. Each element is an expression, evaluated left to right at creation; the stored element is the resulting value.
    a. a. Elements may be literals, arithmetic, variables, or calls.
3. All elements must yield the same data type; that common type is the element type.
    a. Mixed element types are an error.
    b. Any of the six types may serve as element type, including **lis** (nested **lis**) and **glossare**.
        i. homogeneity for **glossare** elements is checked at the type level only (no key-set comparison at creation)
4. Nested **lis** is Pothos's multidimensional form: every element must be of type **lis** and all inner **lis** must share one element type.
5. Length equals the element count at creation and is fixed; no statement or operator adds, removes, or replaces elements. Reassignment creates a whole new **lis** value; it never resizes the old one.



6. **Strina** elements must be quoted **strina** values, either delimiter.

Syntax
<identifier> = [<expr>, <expr>, ... <expr>];
<identifier> = [];

Valid Examples	**Invalid Examples**
nums=[1,2,3];	mixed=[1,'two',3]; (mixed element types)
words=["str",'strs'];	trail=[1,2,]; (trailing comma)
computed=[1+1, square(2)];	unclosed=[1,2; (missing closing bracket)
grid=[[1,2],[3,4]];	box=[[1,2],["a"]]; (nested lis mismatch)

A.5.3 Indexing

1. Elements are accessed with zero-based subscripts <**lis**>[<index>], where the index must be a **wharfa** value (literal or expression).
    a. A **non-wharfa** index is an error.
    b. The valid range is 0 to length−1; any other **wharfa**, including negatives, is an out-of-range runtime error and crashes.
2. A subscript evaluates to a value of the element type and may appear anywhere such a value is expected.
    a. A subscript is not a storage location. Those that're are considered errors.
3. Chained subscripts index nested **lis** left to right.
4. A **lis** value is immutable: elements cannot be changed, added, or removed after creation; the only "change" is reassigning a whole new **lis** of the same element type.
5. Indexing the empty **lis** is always an out-of-range error.
6. **insid** and **lood** read elements but never modify them.

Syntax
<identifier>[<wharfa expr>];
<identifier>[<i>][<j>];



Valid Examples	Invalid Examples
first=nums[0];	x=nums[3]; (out-of-range index)
cell=grid[0][1];	y=nums[1.5]; (non-wharfa index)
belial(nums[1]==2){ prin("two"); }	z=nums["0"]; (non-wharfa index)
val=nums[0];	nums[0]=9; (not storage)
sum=nums[0]+1;	nums[0]+=1; (not storage)
w=nums[0];	w=empty[0]; (empty out-of-range)
n=nums[2];	n=nums[-1]; (negative out-of-range)

A.5.4 Nested lis

1. A **lis** whose elements are all **lis** is a two-dimensional **lis** (its elements are rows); each further level of nesting adds one dimension. The dimensionality equals the depth of bracket nesting in the literal.
    a. Mixing **lis** and **non-lis** elements at one level is an error.
    b. All inner **lis** at the same level must share one element type; an empty inner **lis** is vacuously compatible.
    c. Rows may differ in length; rectangularity is not required, because length belongs to each row value.
2. Initialization writes one pair of square brackets per dimension, outermost first.
3. The number of bracket pairs in the literal equals the number of dimensions; the number of subscripts in a full element access equals the same number.
4. Access uses one subscript per dimension, left to right:
    a. grid[i] yields the row at index i — a **lis** value; grid[i][j] yields the element.
    b. Fewer subscripts than dimensions yields an inner **lis**; more subscripts than the value's dimensions is an error, because some level subscripts a non-collection value.
    c. A subscript applies only to a **lis** (**wharfa** index) or a **glossare** (**strina** key); applying [] to **wharfa**, **flo**, **trua**, or **strina** is an error.
5. Immutability holds at every level:
    a. grid[0][1]=9; and grid[0]=[7,8]; are errors;
    b. only whole-value reassignment of the outermost variable with a matching element type is valid.



6. The element type of a 2D **lis** is "**lis** of X"; == compares recursively and [1,2] **insid** grid tests row equality.
7. A sequence-form **lood** over a 2D **lis** yields rows; the loop variable must be a declared **lis** of the same element type; a nested **lood** visits the elements.
8. **prin**()/**formo**() render nested brackets recursively.
9. A parameter or return declared **lis** carries no dimension or element type in the header; any **lis** may be passed or returned.
    a. Element-type and dimension expectations inside the body, and the element type of a variable declared from a **lis-returning call**, are fixed by the actual value at run time;
    b. a mismatch crashes as a value error, exactly as for the empty **lis**.

Syntax
<identifier> = [[<value>, <value>, ...], ...];
<identifier>[<i>][<j>];

Valid Examples	**Invalid Examples**
grid=[[1,2],[3,4]];	mixed=[[1,2],3]; (lis and wharfa at one level)
ragged=[[1,2,3],[4]];	badrow=[[1,2],["a"]]; (inner element types differ)
cell= grid[0][1];	deep= grid[0][0][0]; (subscripts exceed dimensions)
row= grid[1]; first= row[0];	grid[0][1]= 9; (subscript is not a location)
rv=[0,0]; lood(rv insid grid){ prin(rv[0]); }	t= grid[0]["a"]; (non-wharfa index)
belial([1,2] insid grid){ prin("row"); }	tr= [[1,2],]; (trailing comma)
prin(grid);	s= grid[5][0]; (out-of-range row)

A.6 Glossare

glossare is the key-value mapping type of Pothos:

1. A **glossare** literal is enclosed in curly braces {}. Entries are written as key: value and separated by commas.
    a. A **glossare** literal must contain at least one entry; the empty literal {} is an error.



   i. Unlike **strina** ("") and **lis** ([]), **glossare** has no empty literal; a function with no entries to return must use a sentinel entry or another type.
  b. Braces holding only separators or an incomplete entry ({,}, {"a"}, {: 1}) are errors; a comma immediately before the closing brace is an error.
2. Keys must be a unique **strina** literal.
  a. A duplicate key is an error.
3. Values may be any valid expression; each is evaluated once, in entry order, at creation time, and the stored value is the resulting value.
  a. A value may result in any of the six data types (**strina**, **wharfa**, **flo**, **trua**, **lis**, **glossare**), and one **glossare** may mix value types: {"a": 1, "b": "x"} is valid.
    i. Unlike **lis**, **glossare** has no homogeneity requirement.
  b. **lis** and **glossare** are accepted values (nested collections).
  c. Later changes to variables used in a value expression do not affect the stored value.
4. Keys are static and literal-only.
  a. A variable or expression in the key slot is an error.
5. The mapping is static after creation: keys cannot be added, removed, or renamed, and values cannot be changed.
  a. Entry assignment and key addition are errors; a subscripted entry is not a storage location.
  b. The only change is whole-value reassignment of the variable, which replaces the mapping entirely; any **glossare** value is type-compatible regardless of key set.
  c. Chained subscripts reach into nested collections left to right; each level follows its own subscript rule, **strina** key for **glossare**, **wharfa** index for **lis**.
6. Values are accessed using square brackets: dict["key"].
  a. Accessing a missing key is an error.
  b. Calling a key without quotes is an error.

Syntax
<identifier> = {<strina_key>: <value>, <strina_key>: <value> …};

Valid Examples	**Invalid Examples**
user={"name": "Ana", "score": 95};	x= user["age"]; (missing key)



name= user["name"];	dup={"a": 1, "a": 2}; (duplicate key)
mix={"a": 1, "b": "x"};	v= user[random]; (undeclared key)
store={"nums": [1,2], "inner": {"a": 1}};	emptyDict={}; (empty glossare)
first= store["nums"][0]; deep= store["inner"]["a"];	user["name"]= "Bob"; (entry assignment; static mapping)
base= 1; snap={"n": base, "sum": base+ 1}; base= 9;	user["age"]= 30; (key addition after creation)
	k="a"; d={k: 1}; (non-literal key)
	d={"a": 1,}; (trailing comma)

B. Operators

Operators are symbols and reserved words that combine, compare, or test values to produce a new value:

B.1 Arithmetic Operators:

1. The '+'operator adds two **wharfa**, two **flo** values, or a **wharfa** value with a flo value, or concatenates two **strina** values.
    a. Mixing a **strina** operand with a **wharfa** or **flo** operand is a type error.
2. '-' returns the difference of two **wharfa** or **flo** values; the same symbol also serves as unary negation for negative numbers.
    a. The parser reads '-' as unary negation when it appears in operand position: at the start of an expression, or immediately after '(', ',', '=', an augmented assignment operator, a relational operator, 'reor', '[', '{', ':', or another unary '-'.
    b. The parser reads '-' as binary subtraction when it appears in operator position: immediately after an operand (a literal, an identifier, ')', or ']').
    c. Unary negation applies only to **wharfa** or **flo** operands; negating any other type is an error.
    d. Spacing does not affect the choice: x = -5; and x = - 5; are both negation; x = 5 - 3; and x = 5 -3; are both subtraction.
    e. A '-' immediately followed by another '-' is lexed as the single-line comment start '--', never as double negation;
        i. write '- -5' with a space to negate twice.
3. '*' returns the product of two **wharfa** or **flo** values.



4. '/' returns the integer quotient of two **wharfa** values, discarding decimals, or the decimal quotient of two **flo** values, or of a **wharfa** and a **flo** value, rounded half-up to 4 fractional digits.
5. '%' returns the remainder of two **wharfa** values only.
    a. A **flo** operand or a mixed pair with '%' is a type error; '%' does not promote.
6. Both operands of an arithmetic operation must be numeric (**wharfa** or **flo**); same-type pairs operate in their own type, and mixed pairs operate under the next rule.
    a. A non-numeric operand is an error.
    b. Division by zero is an error.
        i. This applies to both '/' and '%': if the right operand evaluates to zero, it is a runtime error and the program crashes immediately.
        ii. The check occurs at runtime, after both operands are evaluated; a zero arising from **innum**() or from a computation is likewise an error.
        iii. No silent substitute result (such as 0) is ever produced, leading to an error.
7. Mixed **wharfa**/**flo** promotion: when one operand is **wharfa** and the other is **flo** in '+', '-', '*', '/', or any relational operator, the **wharfa** operand is first converted to its exact **flo** equivalent (fixed point, 4 fractional digits); the operation then proceeds as **flo** and produces a **flo** value.
    a. The conversion is exact: every **wharfa** value in the allowed range maps to a representable **flo** value.
    b. Promotion applies only inside operator evaluation. It does not apply to assignment, augmented assignment, argument passing, return values, lis element homogeneity, **insid** operands, or **lood** element checks.
    c. '%' never promotes: both operands must be **wharfa**.

Syntax
<wharfa | flo> + <wharfa | flo>
<wharfa | flo> - <wharfa | flo>
- <wharfa | flo> (unary negation)
<wharfa | flo> * <wharfa | flo>
<wharfa | flo> / <wharfa | flo>
<wharfa> % <wharfa>



Valid Examples	Invalid Examples
x = 7 / 2;	b = 5 / 0; (division by zero)
y = 7 % 2;	g = 5 % 0; (modulo by zero)
n = -5; m = - 5;	q = -"a"; (negation of non-numeric)
d = 5 - 3;	m = 5% 2.0; (mixed modulo)
p = 7.0 / 2.0;	s = 5.0 % 2.0; (modulo not defined for flo)
t = 1 + 1.5;	

B.2 String Concatenation:

1. '+' joins two **strina** values left to right and produces a **strina**.
2. Concatenating a **strina** with any other data type is an error; no implicit conversion exists.

Syntax
<strina> + <strina>

Valid Examples	Invalid Examples
z = "Po" + "thos";	c = "n=" + 5; (concatenation with integer)
full = "Ana" + " Cruz";	a = 5 + "a"; (type mismatch)
chained = "a" + "b" + "c";	v = "s" - "t"; (subtraction of strina)

B.3 Relational Operators:

1. The relational operators '==', '!=', '<', '<=', '>', '>=' compare two values of the same type, or a **wharfa** value with a **flo** value under promotion, and produce a **trua** value.
2. '<', '<=', '>', '>=' order **wharfa** or **flo** values, including mixed pairs; '==' and '!=' test equality of any two values of the same type, or of a **wharfa** and a **flo** (promoted).
    a. Comparing values of different data types, other than the **wharfa**/**flo** numeric pair, is an error.



3. Two **lis** values are equal under == when they have the same length and every pair of corresponding elements is equal; equality recurses into nested **lis**.
    a. A length or element mismatch yields **Inaccu** for == (**Trua** for !=), not an error.
    b. Comparing **lis** of different element types is an error.
    c. Two **glossare** values are equal when they have the same key set and equal values at every key.

Syntax
<same-type> == <same-type>
<same-type> != <same-type>
<wharfa | flo> < | <= | > | >= <wharfa | flo>

Valid Examples	**Invalid Examples**
result = 3 > 2;	d = 3 > "a"; (different types compared)
eq = "a" == "a";	r = [1] < [2]; (ordering defined only for wharfa/flo)
ne = 1 != 2;	[1,2] == ["a","b"] (different element-types)
ord = 1.5 <= 2.5;	[[1,2]] == [["a","b"]]; (nested type-mismatch)
[1,2] == [1,2] (Trua)	[1,2] == [[1,2]]; (mismatched element-types)
[1,2]==[1,2,3] (Inaccu)	
w = 1 == 1.0;	
ord2 = 2> 1.5; (Trua)	

B.4 Logical Operators:

1. The logical operators are the reserved words '**an**' (and), '**othan**' (or), and '**neve**' (not), and require **trua** operands.
    a. A **non-trua** operand is an error.
2. '**an**' produces **Trua** only when both operands are **Trua**.
3. '**othan**' produces **Trua** when at least one operand is **Trua**.
4. '**neve**' negates a single **trua** value.



5. Logical expressions produce a **trua** result.

Syntax
<trua> an <trua>
<trua> othan <trua>
neve <trua>

Valid Examples	**Invalid Examples**
active = Trua; ready = Trua; both = active an ready;	e = 5 an 3; (non-trua operands)
either = active othan Inaccu;	o = Trua othan 1; (non-trua right operand)
neg = neve active;	nv = neve "x"; (negation of non-trua)

B.5 Membership Operator:

1. '<value> **insid** <collection>' tests whether a value is contained in a **lis** or **strina**.
    a. The expression produces **Trua** if the value is found, otherwise **Inaccu**.
2. For a **lis**, the value is compared against each element; for a **strina**, the left operand must be a **strina** contained within the right operand.
    a. Using **insid** with unsupported operand types is an error.
3. '**insid**' also serves as the iteration keyword of sequence-form **lood** headers, membership testing on a **glossare** is not supported.

Syntax
<value> insid <lis | strina>

Valid Examples	**Invalid Examples**
found = 2 insid [1, 2, 3];	f = 2 insid "hello"; (unsupported operands)
sub = "lo" insid "hello";	h = "name" insid user; (glossare membership unsupported)
missing = "z" insid "hello";	i = [1] insid [1, 2]; (left operand type differs from elements)



C. Identifiers

Identifiers are the case-sensitive names given to variables and functions; reserved words are the fixed vocabulary of the language with predefined meanings:

1. An identifier must begin with a letter (a-z, A-Z).
    a. Beginning with a digit or underscore is an error.
2. Succeeding characters may be letters, digits (0-9), or underscores (_) only.
    a. Spaces and special symbols are an error.
3. An identifier has a maximum length of 20 characters.
    a. A minimum of 1 character for variables and 3 characters for functions.
    b. Exceeding 20 characters, or a function name shorter than 3 characters, is a lexical error.
        i. The lexer will accept the first 20 characters and will accept the excess as separate tokens.
4. Global identifiers are case-sensitive and unique. A global variable, constant, or function may be declared only once.
    a. Redeclaring an existing global identifier is an error.
    b. Function parameters are local identifiers. They must be unique within the parameter list of their function.
    c. A parameter may not duplicate the name of a global variable, constant, function, or reserved word.
5. Reserved words cannot be used as identifiers.
6. Reserved words are written only in lowercase letters, except in the truth literals '**Trua**' and '**Inaccu**', which are capitalized and are case-sensitive.
    a. Writing a reserved word in any other casing (e.g, '**Belial**', '**inaccu**') is an error.

Valid Examples	**Invalid Examples**
A	2ndName (starts with a digit)
score	first name (contains a space)
userName	total$ (contains a special symbol)
total_amount	Belial (reserved word as identifier)
add	_age (starts with an underscore)
studentRegNumber2026	studentRegistrationNumber2026 (exceeds 20 characters)



omega = 1; smile = 5;	Ωmega = 1; 😀 = 5; (Non-ASCII characters)

D. Expressions

An expression is a combination of literals, identifiers, operators, and function calls that evaluates to a single value:

1. Every expression evaluates to a value of one of the 6 data types.
2. The value slot of an assignment, argument, condition, or return is fully evaluated before it is stored or used.
3. Parentheses '()' may group sub-expressions; grouped sub-expressions are evaluated first.
4. Every operand must satisfy the type requirements of its operator.
    a. A violation of an operator's type requirement is an error.
5. An expression that cannot be evaluated is an error.

Precedence	**Operator**	**Associativity**
1	() grouping, [] indexing, function call	left
2	unary - (negation), neve (logical NOT) (unary -, neve)	right
3	*, /, %	left
4	+ (addition / concatenation), - (subtraction)	left
5	<, <=, >, >=	left
6	==, !=	left
7	insid	non-associative
8	an	left
9	othan	left

Valid Examples	**Invalid Examples**
grp = (2 + 3) * 4; (grouping first)	d = 5 an 3; (operator type violation)



neg = -3 * 2; (unary binds before *)	e = ghost + 1; (undeclared identifier)
mul = 7 / 2 * 3; (multiplicative, left→right → 9)	f = nums[9] + 1; (out-of-range index)
add = 1 + 2 - 3; (additive, left→right → 0)	g = 5 / 0; (division by zero)
rel = 1 + 1 > 1; (relational after additive)	i = 3 > "a"; (different types compared)
eq = 2 * 2 == 4; (equality after multiplicative)	j = 2 insid "hello"; (unsupported operands)
mem = 2 insid [1, 2, 3] == Trua; (insid binds before ==)	k = (2 + 3; (unbalanced parenthesis)
conj = Trua an 2 > 1; (relational binds before an)	l = neve 5; (neve on non-trua)
disj = Inaccu othan 1 == 1; (== binds before othan)	
h = 1 + 1.5;	

E. Statements and Code Blocks

A statement is a group of reserved words, identifiers, literals, operators, and functions working together to perform one action; a code block groups statements into a single executable unit:

1. Simple statements must end with ';'. Function definitions must end with '};'. Conditional, loop, and branch blocks do not require an additional semicolon after the closing brace.
    a. A missing semicolon is an error.
2. Statements generally start with a reserved word, followed by an identifier, an operator, and a value; some statements may start with an identifier instead.
    a. Simple statements include:
        i. assignment statement
        ii. augmented assignment statement
        iii. **const** declaration statement
        iv. **trua** <identifier>; declaration statement
        v. function call statement
        vi. **reor** return statement
        vii. **paus** break statement
        viii. **persis** continue statement



ix. **pas** no-op statement
x. **deleo** removal statement

3. Anything that returns a value is valid inside the value slot of a statement.
4. A code block is a logical grouping of statements delimited by curly braces '{ }'.
5. A code block executes exclusively under three conditions: when activated by a conditional statement, when activated by a looping statement, or when invoked through a function call.
6. The standalone statement '**pas**;' does nothing.
7. Unbalanced curly braces or parentheses are an error.
8. A comma immediately before a closing], }, or) is an error.

Syntax
<statement>;
{ <statement/s> }

Valid Examples	**Invalid Examples**
prin("Hello");	prin("Hello") (missing semicolon)
x = 1; pas;	{ prin("Hi"); (unbalanced brace)
belial (x == 1) { prin(x); }	prin("Hi"; (unbalanced parenthesis)

F. Comments

Comments are annotations that describe the code but are ignored by the compiler during execution:

1. A single-line comment begins with '--' only when it is not immediately followed by / and ends at the next newline character.
2. A multi-line comment begins with '/--' and ends with '--/'.
    a. A multi-line comment without a closing '--/' is an error (unterminated comment).
    b. A closing '--/' without an opening '/--' is an error.
    c. Nested multi-line comments are not supported. The first --/ closes the comment.
3. Comments may contain ASCII letters, numbers, and special characters, and may appear anywhere in the program.



4. Comments are ignored by the compiler and produce no value.

Syntax
-- <comment>
/-- <comment> --/

Valid Examples	**Invalid Examples**
-- single line comment	/-- missing closer (unterminated comment)
/-- multi line comment --/	/-- a /-- b --/ c --/ (stray --/)
x = 5; -- trailing comment	--/ (missing opening /--)

G. Variables and Constants

Variables and constants are named storage locations that link an identifier to a value:

1. A variable is declared by pairing an identifier with a value using an assignment operator.
2. The variable's type is automatically inferred from its assigned value at declaration and is immutable.
    a. Assigning a value of a different type afterwards is an error; variables cannot be recast.
3. The value slot may hold any literal, expression, function call, or variable, provided it guarantees a value matching the variable's type.
4. Variables must be declared before they are used.
    a. Using an undeclared identifier is an error.
5. A constant is declared with the '**const**' keyword and must be initialized at declaration.
    a. Reassigning a constant is an error.
    b. Deleting a constant with **deleo** is an error.
6. Global variable names, constant names, and function names are unique program-wide.
    a. Function parameter names are local to the function in which they are declared.
    b. A deleted non-constant identifier may be declared again.



Syntax
<identifier> = <value>;
const <identifier> = <value>;
trua <identifier>;

Valid Examples	Invalid Examples
age = 19;	age = 19; age = "nineteen"; (type change)
name = "Ana";	const LIMIT; (uninitialized constant)
const LIMIT = 100;	x = y + 1; (y undeclared)
total = age + 1;	LIMIT = 200; (constant reassignment)

H. Assignment and Augmented Assignment

Assignment stores an evaluated value into a variable: augmented assignment combines an arithmetic operation with assignment in one statement:

1. The assignment operator is '='; the right-hand side is evaluated first before being stored in memory.
2. The augmented operators are '+=', '-=', '*=', '/=', and '%='.
    a. 'x += 1;' is equivalent to 'x = x + 1;'
    b. %= applies only to **wharfa** variables
3. The resulting value must match the existing type of the variable.
    a. for a **lis** variable, matching means the same element type; the new value's length may differ.
    b. Promotion never crosses the assignment: an augmented operation whose promoted result is **flo** cannot be stored into a **wharfa** variable, and vice versa.
4.
5. Augmented assignment may only be applied to an already declared variable.
    a. Applying it to an undeclared identifier is an error.
6. Any assignment or augmented assignment to a constant is an error.



Syntax
<identifier> <assops> <value>; (where <assops> is '=', '+=', '-=', '*=', '/=', '%=')

Valid Examples	Invalid Examples

a = 5; b = 2;
c = 10; d = 3;
e = 8; f = 5;
	g -= 1; (g undeclared)

b += 3; c -= 2; d*= 4;	s = "X"; s -= 1; (type-incompatible operation)
e /= 2; f %= 3;	const K = 1; K += 1; (constant modification)
	i = 1; i+= 1.5; (promoted wharfa)

I. Built-In Functions

Built-in functions are always available without declaration, and their names are reserved:

I.1 prin()

1. '**prin**()' prints values to standard output.
2. Arguments may be literals, variables, expressions, or function calls.
3. String arguments must be quoted **strina** literals (double or single quotes).
4. The arguments must be enclosed in parentheses, and the statement must end with a semicolon ';' just outside them.
    a. Missing parentheses or semicolon is an error.

Syntax
prin(<expression>);

Valid Examples	Invalid Examples
prin("Hello, World!");	prin"Hello"; (missing parentheses)
score = 90; prin(score);	prin(score; (unbalanced parenthesis)
prin(a + b); prin (1 + 1);	prin(Hello); (undeclared identifier/unquoted string literal)



I.2 inpub()

1. '**inpub**()' reads one line of user input from the console.
2. **inpub**() always returns a **strina** value.
3. The returned value may be assigned to a variable. The variable's inferred type is **strina**.
4. **inpub**() must be called with empty parentheses and exact casing.
    a. **inpub**(or **Inpub**() is an error.

Syntax
<identifier> = inpub();

Valid Examples	**Invalid Examples**
name = inpub();	Parenthesis = inpub(; (unbalanced casing)
capitalization = inpub();	capitalization = Inpub(); (wrong casing)

I.3 innum()

1. '**innum**()' reads one line of user input from the console.
2. **innum**() always returns a **wharfa** value.
3. The input must be a valid **wharfa** integer.
    a. If the input contains non-integer characters, it is a runtime error.
    b. If the value is outside the range -99999999 to 99999999, it is a runtime error.
4. **innum**() must be called with empty parentheses and exact casing.
    a. **innum**(or **Innum**() is an error.

Syntax
<identifier> = innum();

Valid Examples	**Invalid Examples**
age = innum();	x = innum(; (unbalanced parenthesis)
score = innum();	y = Innum(); (wrong casing)
prin(age + 1);	z = innum(); (strina input)



I.4 inflo()

1. **inflo**() reads one line of user input from the console.
2. **inflo**() always returns a **flo** value.
3. The input must be a valid **flo** literal: an optional leading -, then at least one digit on both sides of a single decimal point; integer part capped at 8 significant digits, fractional part at 4 digits.
    a. Non-numeric characters, a missing digit on either side of the point, or a second point are runtime errors.
    b. Input exceeding the integer or fractional caps is a runtime error.
    c. A leading + is a runtime error.
    d. Integer-shaped input (e.g. 5) is a runtime error: each reader accepts precisely the literal shape of its return type.
    e. Leading zeros in the integer part are discarded; the value is stored as fixed-point with 4 fractional digits.
4. The returned value may be assigned to a variable; the variable's inferred type is **flo**.
5. **inflo**() must be called with empty parentheses and exact casing.
    a. **inflo**(or **Inflo**() is an error.

Syntax
<identifier> = inflo();

Valid Examples	**Invalid Examples**
price = inflo();	p = inflo(; (unbalanced parentheses)
rate = inflo();	P = Inflo(); (wrong casing)
t = inflo(); prin(t + 1.0);	input: 5, .5, 5., 1.23456, +2.5 (runtime errors)

I.5 formo()

1. '**formo**()' formats a **strina** by substituting placeholders with the values of variables.
2. **formo**() must be called with parentheses enclosing a quoted **strina** literal.
    a. Writing **formo** without parentheses is an error.
3. Placeholders are written {<identifier>} inside the format string, and each must name a declared variable.
    a. A placeholder naming an undeclared variable is an error.



b. Only curly braces interpolate: '( )' and '[ ]' around a name remain a literal text.
c. Inside a **formo**() format string, {{ produces a literal { and }} produces a literal }.

4. **formo**() returns a **strina** and may appear on the right-hand side of an assignment, as an argument to **prin**(), or anywhere a **strina** value is expected.

Syntax
formo("<text {identifier} text>");

Valid Examples	**Invalid Examples**
varx = "Sean"; vary = 19; greet = formo("I am {varx}, age {vary}.");	greet = formo"I am {varx}."; (missing parentheses)
varz = "Charles"; hello = formo("I am {varz}. {{literal}}");	greet = formo("I am {varx}, age {vary}.; (unterminated string)
prin(formo("I am {varx}, age {vary}."));	greet = formo("I am {nobody}."); (undeclared placeholder)

I.6 Display Forms

1. **prin**() and **formo**() render values by type:
    a. **wharfa**: normalized digits (leading zeros discarded), - prefix when negative.
    b. **flo**: 4 fractional digits, trailing zeros trimmed, at least one kept.
    c. **trua**: **Trua** / **Inaccu**.
    d. **Strina** (standalone argument): raw characters with escapes resolved, no surrounding quotes.
    e. **lis**: elements rendered by these rules, separated by ', ', enclosed in []; **strina** elements inside a collection appear in double quotes with internal escapes (\", \\, ...); the empty **lis** renders [].
    f. **glossare**: entries as "key": value in key-insertion order, separated by ', ', enclosed in {}.
    g. Nested collections render recursively under the same rules.
2. A **formo**() placeholder interpolates exactly the rendering **prin**() would produce for that variable (so a **strina** variable interpolates raw, but a **strina** element inside an interpolated **lis** keeps its quotes).
3. Display forms are output text only; they are not values or expressions in the program.



Input	Output
prin(100);	100
prin(3.50);	3.5
is_true = Trua; prin(is_true);	Trua
prin("hello");	hello
prin([1,2,3]);	[1, 2, 3]
prin([[1,2],[3,4]]);	[[1, 2], [3, 4]]
prin([]);	[]
prin({"n":1,"s":"x"});	{"n": 1, "s": "x"}
formo("v={nums}"); nums=[1,2]	v=[1, 2]

J. Conditional Statements

Conditional statements allow the program to choose which code block to execute based on a condition:

1. A conditional statement begins with '**belial**'.
2. The condition must be enclosed in parentheses.
3. The condition must evaluate to a '**trua**' value.
    a. A condition of any other type is an error.
4. The body must be enclosed in curly braces '{ }'.
5. Additional branches may be written using '**belsa**', each requiring its own condition in parentheses.
6. A final optional fallback branch uses '**elsa**', which takes no condition.
    a. Giving '**elsa**' a condition, or placing any branch after '**elsa**', is an error.
7. Conditional blocks may be nested.
8. Statements inside the block must end with a semicolon ';'.



Syntax

belial (<condition>) {
<statement/s>
} belsa (<condition>) {
<statement/s>
} belsa (<condition>) {
<statement/s>
} elsa {
<statement/s>
}

Valid Examples	**Invalid Examples**
score = 75; belial (score >= 90) { prin("Excellent"); } belsa (score >= 75) { prin("Good"); } elsa { prin("Needs improvement"); }	belial score > 70 { prin("Pass"); } (missing parentheses)

inventory = ["apple", "banana", "cherry"];
target = "banana";
 	
belial (target insid inventory) {
  prin("Item is in stock.");
}
	belial (score > 70) prin ("Pass"); (missing braces)

role = "admin";
active = Trua;
belial (active an role == "admin") {
  prin("Admin panel.");
} belsa (active an role == "editor") {
  prin("Editor tools.");
} elsa {
  prin("Read only.");
}
	belial (score) { prin("Pass"); } (condition not trua)

age = 16;
has_id = Trua;
belial (age >= 18) {
  belial (has_id) {
  prin("Entry allowed.");
  } elsa {
  prin("ID required.");
	elsa (score >= 74) { prin("Fail"); } (elsa condition)



}
} elsa {
prin("Students only.");
}	

K. Looping Statements

Loops repeat a code block, either while a condition holds '**cycla**' or once per element of an iterable '**lood**':

K.1 Cycla Loop

1. A '**cycla**' loop begins with the reserved word '**cycla**'.
2. The condition must be enclosed in parentheses and must evaluate to a '**trua**' value.
    a. A **non-trua** condition is an error.
3. The loop body must be enclosed in curly braces '{ }'.
4. The condition is checked before each iteration.
5. The loop continues while the condition is '**Trua**', and stops when it becomes '**Inaccu**'.
6. The loop body may contain any valid statement, including:
   a. assignment, augmented assignment, **const**, and **trua** declaration statements;
   b. function call statements (**prin**, **inpub**, **formo**, **innum**, or **user-defined**);
   c. conditional statements (**belial**/**belsa**/**elsa**), which may nest inside the loop;
   d. other looping statements (**cycla** or **lood**); loops may be nested;
   e. loop control statements (**paus**, **persis**), which act on the nearest enclosing loop (L);
   f. the no-op statement **pas**;.
7. A bare condition is not a statement; a condition may appear only inside the parentheses of a **belial**, **belsa**, **cycla**, or **lood** header.
8. **reor** may appear inside a loop body only when the loop itself is inside a function body.
9. Pothos has no increment (++) or decrement (--) operators. These operators must instead be written as i += 1, and i -= 1 respectively for **cycla**.

Syntax
cycla (<condition>) { <statement/s> }

Valid Examples	**Invalid Examples**



count = 5; cycla (count > 0) { prin(count); count -= 1; }	cycla count > 0 { count -= 1; } (missing parentheses)

count = 3;
i = 0;
cycla (count > 0) {
  lood (i insid [1, 2]) {
  prin(i);
  }
  belial (count == 2) {
  paus;
  }
  count -= 1;
}
	cycla (count > 0) count -= 1; (missing braces)

K.2 Lood Loop

1. A '**lood**' loop begins with the reserved word '**lood**'.
2. The loop variable is placed before the reserved operator '**insid**', and the iterable expression after it.
3. The variable placed before '**insid**' must already be initialized (declared) before the loop.
    a. An uninitialized loop variable is an error.
4. The loop header must be enclosed in parentheses, and the loop body in curly braces '{ }'.
5. The loop variable receives each element from the iterable in order, one per iteration.
6. The loop variable follows identifier rules.
7. The iterable may be a '**lis**', '**strina**', '**glossare**'.
    a. A '**lis**' yields its elements.
    b. A '**strina**' yields its characters as one-character '**strina**' values.
    c. A '**glossare**' yields its values in the order their corresponding keys appear in the literal.
8. All iterated elements must be of the same data type as the loop variable.
    a. An element whose type differs from the loop variable's type is an error.
9. A **lood** loop also accepts a counted header: **lood**(<init>; <condition>; <step>){}.
    a. All three clauses are required; a missing clause or a missing/extra ';' delimiter is an error.
    b. The init clause is a simple assignment executed once before the first iteration; it declares the variable if undeclared (type inferred) or reassigns an existing same-type variable.



c. The condition clause must evaluate to a **trua** value and is checked before each iteration.
d. The step clause is an assignment or augmented assignment executed after each iteration of the body, including when **persis** is executed.
e. **paus** exits the loop; **persis** skips to the step clause.
f. A variable created by the init clause follows normal declaration rules and remains accessible after the loop.
g. Pothos has no increment (++) or decrement (--) operators. These operators must instead be written as i += 1, and i -= 1 respectively for **lood** loop.

10. The loop body may contain any valid statement, including:
    a. assignment, augmented assignment, **const**, and **trua** declaration statements;
    b. function call statements (**prin**, **inpub**, **formo**, **innum**, or **user-defined**);
    c. conditional statements (**belial**/**belsa**/**elsa**), which may nest inside the loop;
    d. other looping statements (**cycla** or **lood**); loops may be nested;
    e. loop control statements (**paus**, **persis**): **paus** exits the nearest enclosing loop; **persis** skips to the step clause in counted form or to the next element in sequence form;
    f. the no-op statement **pas**;
11. A bare condition is not a statement; a condition may appear only inside the parentheses of a **belial**, **belsa**, **cycla**, or **lood** header, or in the condition clause of a counted header.
12. **reor** may appear inside a loop body only when the loop itself is inside a function body.
13. An empty iterable ([], "") performs zero iterations; the loop variable keeps its previous value and no error occurs.

Syntax

lood (<loopvar> insid <iterable>) {
  <statement/s>
}

lood (<init>; <condition>; <step>) {
  <statement/s>
}

Valid Examples	**Invalid Examples**
nums = [1, 2, 3]; m = 0;	lood n insid nums { prin(n); } (missing



lood (m insid nums) { prin(m); }	parentheses, uninitialized n)
word = "hi"; ch = "h"; lood (ch insid word) { prin(ch); }	m = 0; lood (m insid undeclaredList) { } (undeclared iterable)

sum = 0;
lood (i = 1; i <= 4; i += 1) {
  sum += i;
}
prin(sum);
	lood(i=0; 5; i–){
	} (condition not trua)

scores = {"ana": 90, "ben": 85};
val = 0;
lood (val insid scores) {
  prin(val);
}
	lood(j+=1; j<3; j++){
	}
	(augmented assign on undeclared identifier)

L. Loop Control Statements

Loop control statements alter the normal flow of the nearest enclosing loop:

1. '**paus**;' immediately exits the nearest enclosing loop.
2. '**persis**;' skips the remaining statements in the current iteration and proceeds to the next iteration.
3. '**paus**' and '**persis**' are valid only inside loops.
    a. Using either outside a loop body is an error.
4. Loop control statements must end with ';'.
5. '**paus**' and '**persis**' may also appear inside a conditional statement (**belial**/**belsa**/**elsa**) body, provided that conditional is enclosed by a loop, as in Python's conditional break/continue.
    a. When executed inside a conditional, the statement still targets the nearest enclosing loop. The conditional block itself is never a target. Its only difference from direct use in the loop body is that it runs conditionally.
    b. A '**paus**' or '**persis**' inside a conditional that is not enclosed by any loop will result in an error.

Valid Examples	**Invalid Examples**
cycla (x < 10) { x += 1;	cycla (x < 10) { x += 1; } paus; (loop control outside a loop)



belial (x == 5) { paus; }
prin(x);
}	

lood (m insid nums) {
belial (m == 0) { persis; }
prin(m);
}
	paus (missing semicolon)

x = 0;
cycla (x < 10) {
  x += 1;
  belial (x > 4) {
  belial (x == 7) {
  paus;
  }
  }
}
prin(x);
	x = 0;
	belial (x == 5) {
	  paus;
	} (not inside a loop)

M. Functions

A function is a reusable, named block of code that is declared once and invoked by name:

M.1 Function Declaration

1. A function declaration is initiated with an explicit return type (a data type keyword), immediately followed by the function name.
    a. A missing or non-type return keyword is an error.
    b. List of allowed data type keywords:
        i. **wharfa**
        ii. **flo**
        iii. **strina**
        iv. **trua**
        v. **lis**
        vi. **glossare**
2. The function name must follow identifier rules, with a minimum of three characters.
3. The parameter list is enclosed in parentheses; a function with no parameters uses empty parentheses.
4. The parameter list contains only comma-separated <type> <identifier> pairs.



   a. A '=' inside the parameter list is a syntax error; default parameter values are not supported. Because they are not supported, neither an implicit-typed default nor an explicit-typed default is valid; both are syntax errors.
   b. Because no defaults exist, every call must supply exactly as many arguments as the function has parameters.
5. The function body must be enclosed in curly braces { }, and the declaration ends with ';' after the closing brace.
6. Each parameter is local to the function body. It is visible only inside that function.
7. A function must be declared before it is called.
   a. Calling an undeclared function is an error.
8. A function identifier may be declared only once.

Syntax

<return_type> <identifier>(<type> <parameter>, <type> <parameter>, ...){
  <statement/s>;
};

Valid Examples	**Invalid Examples**

wharfa square(wharfa n){
  reor n * n;
};
	prin("Hello") { prin("Invalid"); }; (no return type)

flo half(flo n){
  reor n / 2.0;
};
	flo half(flo n) { reor n/2; }; (wharfa return in flo function)

strina greet(){
  reor "Hi";
};
	wharfa ab(a) { reor a; }; (function name under 3 characters)

wharfa myFunc(wharfa x){
  reor x;
};
	wharfa myFunc(wharfa x= 5){ reor x; }; (explicit-typed default)

wharfa myFunc(wharfa x){
  reor x;
};
prin(myFunc(5));
	wharfa myFunc(x= 5){ reor x; }; (implicit-typed default)



M.2 Parameters

1. Each parameter consists of a data type and an identifier.
2. A function may have zero or more parameters.
3. Multiple parameters must be separated by commas.
4. Parameters follow identifier rules and serve as the names through which the body receives argument values.
5. At call time, arguments must match parameters in count, order, and type.
    a. A mismatch in argument count is an error.
    b. Promotion does not apply at the call boundary: a **wharfa** argument for a **flo** parameter, or a **flo** argument for a **wharfa** parameter, is an error.
6. Each argument must be a literal, variable, or expression that guarantees a value.
7. A parameter declaration is exactly one data-type keyword followed by one identifier; nothing else may appear in the pair.
8. In a call, argument slots accept expressions only. Assignment and augmented assignment are statements and are never valid arguments.
    a. Assignment appears only in the init/step clauses of a counted **lood** header, never in a function header or argument list.

M.3 Return Values

1. '**reor** <expression>;' returns a value and immediately exits the function.
2. If an explicit return type is given, the returned value must match that type.
    a. A returned value of a different type is an error.
    b. Promotion does not apply to **reor**: a **wharfa** expression in a **flo-returning** function, or vice versa, is an error.
3. If a function is declared with return type **wharfa**, **reor** must return a **wharfa** value. The same applies to **flo**, **strina**, **trua**, **lis**, and **glossare**.
4. A function that reaches the end of its body without returning a value produces an error.
5. '**reor**' may appear only inside a function body.
    a. Using **reor** outside a function is an error.

M.4 Function Calls

1. To invoke a function, its unique name must be stated along with its arguments, separated by commas and contained within parentheses '( )'.
2. A function call may appear anywhere a value of its return type is expected: value slots, conditions, output, and other expressions.
3. Arguments are evaluated before the function body executes.



Syntax
<identifier>(<argument/s>);

Valid Examples	Invalid Examples
prin(square(4));	total = square(3; (unbalanced parenthesis)
msg = greet();	mystery(); (function not declared)
total = square(2) + 1;	square("Word"); (strina argument passed to wharfa parameter)
half(1.0);	half(1); (wharfa argument to flo parameter)
x = 5; myFunc(x);	myFunc(x= 5); (assignment as argument)

M.5 Recursion

1. A function may call itself recursively.
2. The maximum recursion depth is 999.
    a. Exceeding 999 nested calls is an error.

Valid Examples	Invalid Examples

wharfa factorial(wharfa n){
  belial(n <= 1){
  reor 1;
  }
  elsa{
  reor factorial(n - 1) * n;
  }
};
	wharfa endless(wharfa n){
	  reor endless(n + 1);
	}; (Runtime Error)

M.6 Scope and Visibility

1. Usable variables and functions must be declared first to be used, in top-to-bottom order.
2. Functions can access variables defined anywhere in the program, provided the variable has been declared by the time the function is called.
    a. Accessing a variable that is declared only after the call is an error.
3. Global identifiers are unique across the whole program. Parameters are local to their function.



   a. Reusing a global variable, constant, or function name in another block, loop, or function is an error.
   b. Reusing a parameter name in another function is allowed, provided no global identifier has that name.
4. Declarations remain accessible anywhere after their point of declaration, until deleted.

N. Object Removal

1. '**deleo**' removes a declared identifier and its linked value from the program:
2. '**deleo**' must be followed by an identifier.
   a. Using **deleo** on a literal or non-identifier token is an error.
3. The identifier must exist at the point of deletion.
   a. Deleting an undeclared identifier is an error.
4. The statement must end with ';'.
5. Using a deleted identifier after deletion is invalid.
   a. Any reference to a deleted identifier is an error.
6. Constants cannot be deleted or redeclared.
   a. **deleo** applied to a constant is an error.
7. After **deleo** <identifier>;, the identifier is removed from the active symbol table. The name may be declared again unless the identifier is a constant.

Syntax
deleo <identifier>;

Valid Examples	**Invalid Examples**
temp = 1; deleo temp;	deleo ghost; (undeclared identifier) deleo 5; (not an identifier) const K = 1; deleo K; (deleting a constant) x = 1; deleo x; prin(x); (reference after deletion)



VII. REGULAR DEFINITIONS

whitespace	{space, newline, tab}
caplet	{ A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z }
lowlet	{ a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z }
letter	{ caplet, lowlet }
digit	{ 1, 2, 3, 4, 5, 6, 7, 8, 9 }
escape	{\", \' , \\ , \n , \t, \0 }
numbers	{ 0, digit }
alphanum	{ letter, numbers}
mathop	{ +, -, *, /, % }
relop_ch	{ <, >, ==, ! }
assops	{=, +=, -=, *=, /=, %=}
ascii	{ any printable ascii character }
ml_char	{ascii, whitespace}
eof	{ end-of-file marker }
special characters	{unicode symbols not found on ascii table}
delim_space	space, tab
delim_quote	', "
delim_datatype	delim_space, newline
delim_i	delim_space, (
delim_bool	delim_space,), },], ;, , , :, mathop, relop_ch, eof
delim_logic	delim_space, (
delim_block	delim_space, newline, {
delim_stmt	delim_space, newline, ;



delim_struc	delim_space, newline, ; , (, [, {, -
delim_deleo	delim_space
delim_const	delim_space
delim_insid	delim_space, newline, (,{, [, “ , ‘ , -
delim_plus	delim_space, letter, numbers, delim_quote, (
delim_arithmetic	delim_space, letter, numbers, (
delim_oper	delim_space, letter, numbers, (, -
delim_equality	delim_space, delim_quote, letter, numbers, (, -
delim_equal	delim_space, letter, numbers, (, [, {, -
delim_augoper	delim_space, letter, numbers, delim_quote, (, -
delim_openparen	delim_space, letter, numbers, delim_quote, (, [, -
delim_closeparen	delim_space, newline, relop_ch,),] ,{, }, ; , ,
delim_openbracket	delim_space, letter, numbers, delim_quote, (, [, {, -
delim_closebracket	delim_space, newline, relop_ch,),] ,}, ; , ,
delim_opencurly	delim_space, letter, numbers, delim_quote
delim_closecurly	delim_space, delim_quote, newline, relop_ch,),] ,}, ;
delim_comma	delim_space, letter, numbers, delim_quote, (, [, {
delim_dqopen	delim_space, ascii
delim_dqclose	delim_space, newline, , ,) ,] , } , ;, : , relop_ch, mathop
delim_sqopen	delim_space, ascii
delim_sqclose	delim_space, newline, , ,) ,] , } , ; , : , relop_ch, mathop
delim_colon	delim_space, newline, letter, numbers, delim_quote, - , [, {
delim_scolon	delim_space, letter, } , -, /, eof



delim_slcomment	delim_space, ascii, special characters
delim_mlcomment_close	delim_space, letter, special characters, newline, }, eof, /
delim_id	delim_space, newline, mathop, assops, relop_ch, [,], },), ;
delim_num	delim_space, newline, mathop, relop_ch, ; ,], },)



VIII. REGULAR EXPRESSION

A. Reserved Words

Pothos	REGULAR EXPRESSION	TOKEN
1. Built-in Data Types		
strina	(s)(t)(r)(i)(n)(a)	strina
wharfa	(w)(h)(a)(r)(f)(a)	wharfa
flo	(f)(l)(o)	flo
trua	(t)(r)(u)(a)	trua
lis	(l)(i)(s)	lis
glossare	(g)(l)(o)(s)(s)(a)(r)(e)	glossare
2. Input / Output Functions		
prin()	(p)(r)(i)(n)	prin
inpub()	(i)(n)(p)(u)(b)	inpub
formo()	(f)(o)(r)(m)(o)	formo
innum()	(i)(n)(n)(u)(m)	innum
inflo()	(i)(n)(f)(l)(o)	inflo
3. Value & Truth Literals		
Inaccu	(I)(n)(a)(c)(c)(u)	Inaccu
Trua	(T)(r)(u)(a)	Trua
4. Logical Operators		
an	(a)(n)	an
othan	(o)(t)(h)(a)(n)	othan
neve	(n)(e)(v)(e)	neve
5. Structural Control Flow		
belial	(b)(e)(l)(i)(a)(l)	belial



belsa	(b)(e)(l)(s)(a)	belsa
elsa	(e)(l)(s)(a)	elsa
lood	(l)(o)(o)(d)	lood
cycla	(c)(y)(c)(l)(a)	cycla
paus	(p)(a)(u)(s)	paus
persis	(p)(e)(r)(s)(i)(s)	persis
pas	(p)(a)(s)	pas
6. Structural Definitions		
reor	(r)(e)(o)(r)	reor
7. Variable Operations		
deleo	(d)(e)(l)(e)(o)	deleo
const	(c)(o)(n)(s)(t)	const
8. Object & Collection Membership		
insid	(i)(n)(s)(i)(d)	insid

B. Reserved symbols

Pothos	Regular Expressions	Token
Arithmetic Operators		
+	(+)	+
-	(-)	-
*	(*)	*
/	(/)	/
%	(%)	%
Relational Operators		
<	(<)	<
>	(>)	>



<=	(<)(=)	<=
>=	(>)(=)	>=
==	(=)(=)	==
!=	(!)(=)	!=
Assignment and Augmented Operators		
=	(=)	=
+=	(+)(=)	+=
-=	(-)(=)	-=
*=	(*)(=)	*=
/=	(/)(=)	/=
%=	(%)(=)	%=
Grouping Symbols		
(	(()	(
)	())	)
[	([)	[
]	(])	]
{	({)	{
}	(})	}
Others		
,	(,)	,
"	(")	"
'	(')	'
:	(:)	:
;	(;)	;
--	(-)(-)	--



/--	(/)(-)(-)	/--
--/	(-)(-)(/)	--/

C. Reserved Literals

Literal	**Regular expressions**	**Token**
identifier	letter(alphanum|_)^{19}	id
integer	(0) | (digit)(numbers)^{7}	wharfa_lit
float	((0) | (digit)(numbers)^{7}) (.) (numbers)^{4}	flo_lit
string	"ascii*" | 'ascii*'	strina_lit
boolean	(T)(r)(u)(a) | (I)(n)(a)(c)(c)(u)	trua_lit
single-line comment	(-)(-)((ascii - {/})(ascii)*)?	comment_sl
multi-line comment	(/)(-)(-)(ml_char)*(-)(-)(/)	comment_ml



IX. DELIMITERS

A. RESERVED WORDS

RESERVED WORDS	DELIMITER	DEFINITION
Built-in Data Types		
strina	{delim_space, newline}	delim_datatype
wharfa	{delim_space, newline }	delim_datatype
flo	{delim_space, newline }	delim_datatype
trua	{delim_space, newline}	delim_datatype
lis	{delim_space, newline}	delim_datatype
glossare	{delim_space, newline}	delim_datatype
Input / Output Functions		
prin	{ delim_space, (}	delim_io
inpub	{ delim_space, (}	delim_io
formo	{ delim_space, (}	delim_io
innum	{ delim_space, (}	delim_io
inflo	{ delim_space, (}	delim_io
Value & Truth Literals		
Inaccu	{ delim_space,), },], ;, , , :, mathop, relop_ch, eof}	delim_bool
Trua	{ delim_space,), },], ;, , , :, mathop, relop_ch, eof}	delim_bool
Logical Operators		
an	{ delim_space, (}	delim_logic
othan	{ delim_space, (}	delim_logic
neve	{ delim_space, (}	delim_logic
Structural Control Flow		



belial	{ delim_space, (}	delim_logic
belsa	{ delim_space, (}	delim_logic
elsa	{ delim_space, newline, { }	delim_block
lood	{ delim_space, (}	delim_logic
cycla	{ delim_space, (}	delim_logic
paus	{ delim_space, newline, ; }	delim_stmt
persis	{ delim_space, newline, ; }	delim_stmt
pas	{ delim_space, newline, ; }	delim_stmt
Structural Definitions		
reor	{ delim_space, newline, ; (, [, {, - }	delim_struc
Variable Operations		
deleo	{ delim_space}	delim_deleo
const	{ delim_space }	delim_const
Object & Collection Membership		
insid	{ delim_space, newline, (, [, “ , ‘ , - }	delim_insid

B. RESERVED SYMBOLS

Reserved Symbols	Definition	Delimiter
Arithmetic Operators		
+	{ delim_space, letter, numbers, delim_quote, (}	delim_plus
-	{ delim_space, letter, numbers, (}	delim_arithmetic
*	{ delim_space, letter, numbers, (}	delim_arithmetic
/	{ delim_space, letter, numbers, (}	delim_arithmetic



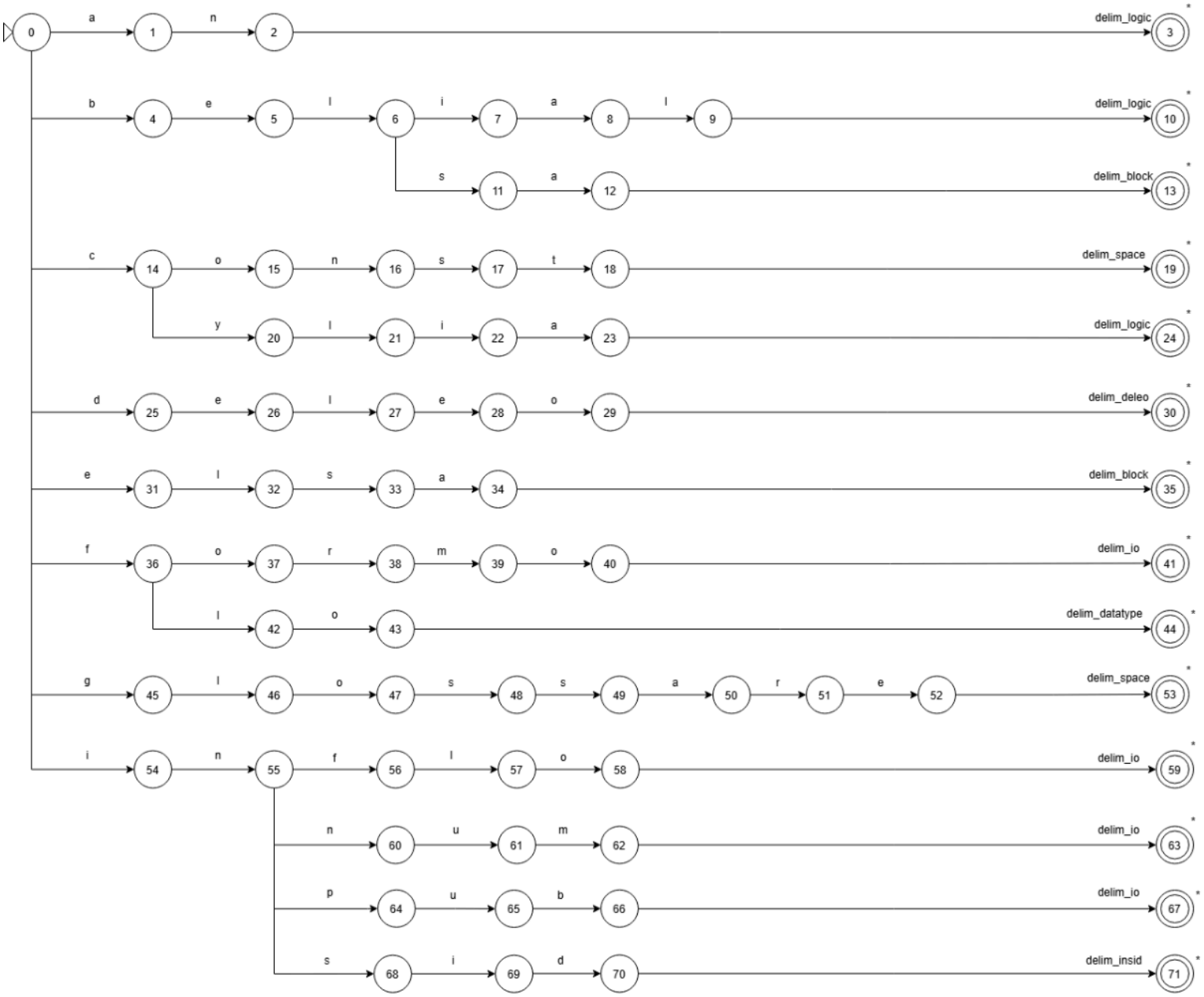
%	{ delim_space, letter, numbers, (}	delim_arithmetic
Relational Operators		
<	{ delim_space, letter, numbers, (, - }	delim_oper
>	{ delim_space, letter, numbers, (, - }	delim_oper
<=	{ delim_space, letter, numbers, (, - }	delim_oper
>=	{ delim_space, letter, numbers, (, - }	delim_oper
==	{ delim_space, delim_quote, letter, numbers, (, - }	delim_equality
!=	{ delim_space, letter, numbers, (, - }	delim_oper
Assignment and Augmented Operators		
=	{ delim_space, letter, numbers, (, [, {, - }	delim_equal
+=	{ delim_space, letter, numbers, (, - }	delim_augoper
-=	{ delim_space, letter, numbers, (, - }	delim_augoper
*=	{ delim_space, letter, numbers, (, - }	delim_augoper
/=	{ delim_space, letter, numbers, (, - }	delim_augoper
%=	{ delim_space, letter, numbers, (, - }	delim_augoper
Grouping Symbols		
(	{delim_space, letter, numbers, delim_quote, (, [, { , -}	delim_openparen
)	{delim_space, newline, relop_ch,),], {, }, ;, ,, mathop}	delim_closeparen

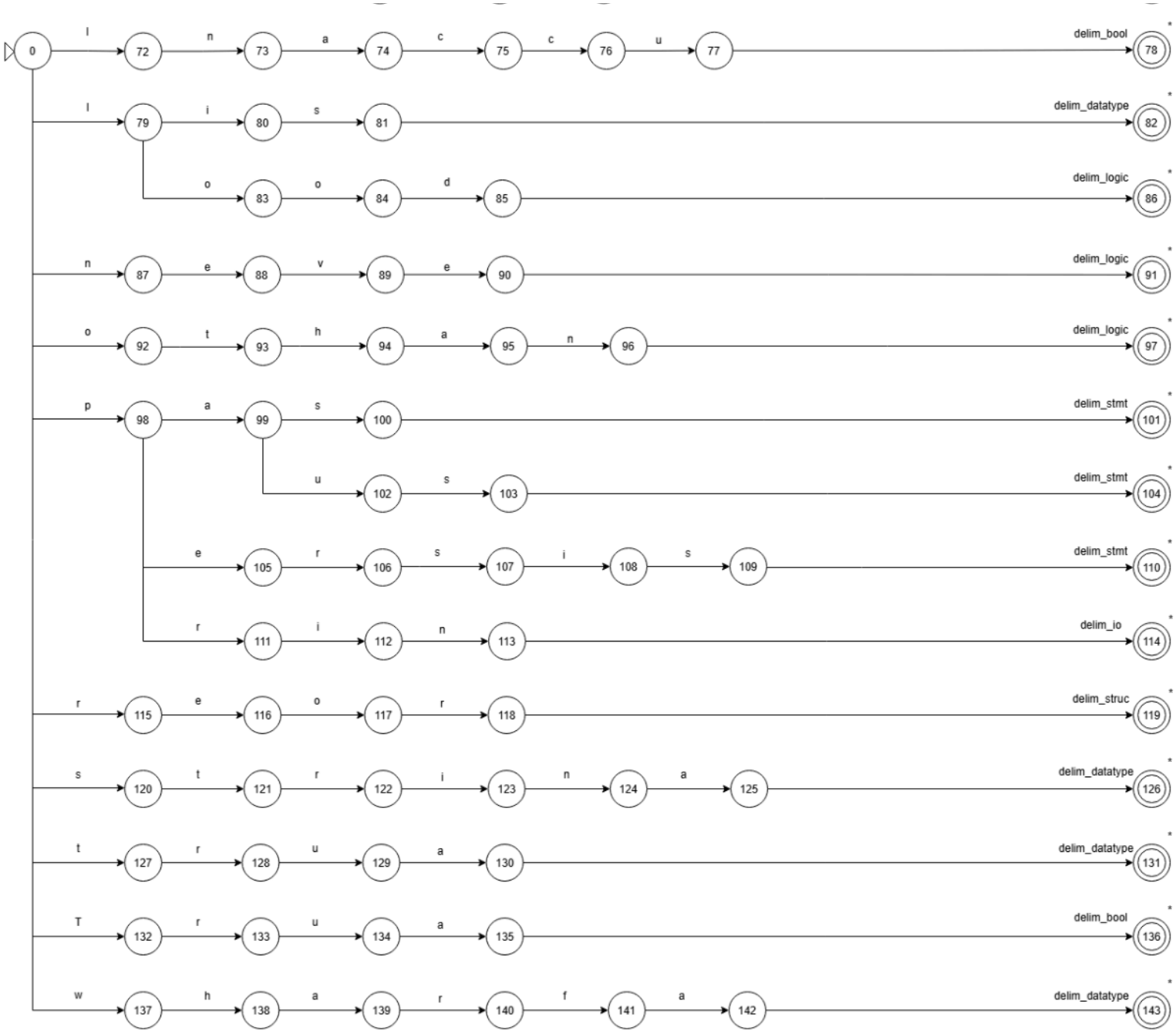


[	{delim_space, letter, numbers, delim_quote, (, [, {, - }	delim_openbracket
]	{delim_space, newline, relop_ch,), [,], }, ;, ,, mathop}	delim_closebracket
{	{delim_space, letter, numbers, delim_quote }	delim_opencurly
}	{delim_space, delim_quote, newline, relop_ch,),], }, ;, mathop}	delim_closecurly
Others		
,	{delim_space, letter, numbers, delim_quote, (, [, {, -}	delim_comma
" (opening)	{delim_space, ascii }	delim_dqopen
" (closing)	{delim_space, newline, , ,) ,] , } , ;, : , relop_ch, mathop }	delim_dqclose
' (opening)	{delim_space, ascii }	delim_sqopen
' (closing)	{delim_space, newline, , ,) ,] , } , ; , : , relop_ch, mathop }	delim_sqclose
:	{delim_space, newline, letter, numbers, delim_quote, - , [, { }	delim_colon
;	{delim_space, letter, } , -, /, eof }	delim_scolon
--	{delim_space, ascii, special characters}	delim_slcomment
/--	{ delim_space, ascii, special characters, newline }	delim_mlcomment_open
--/	{ delim_space, letter, special characters, newline, }, eof, / }	delim_mlcomment_close
Identifiers		
{delim_space, newline, mathop, assops, relop_ch, [,], },), ; }		delim_id
{delim_space, newline, mathop, relop_ch, ; ,], },) }		delim_num

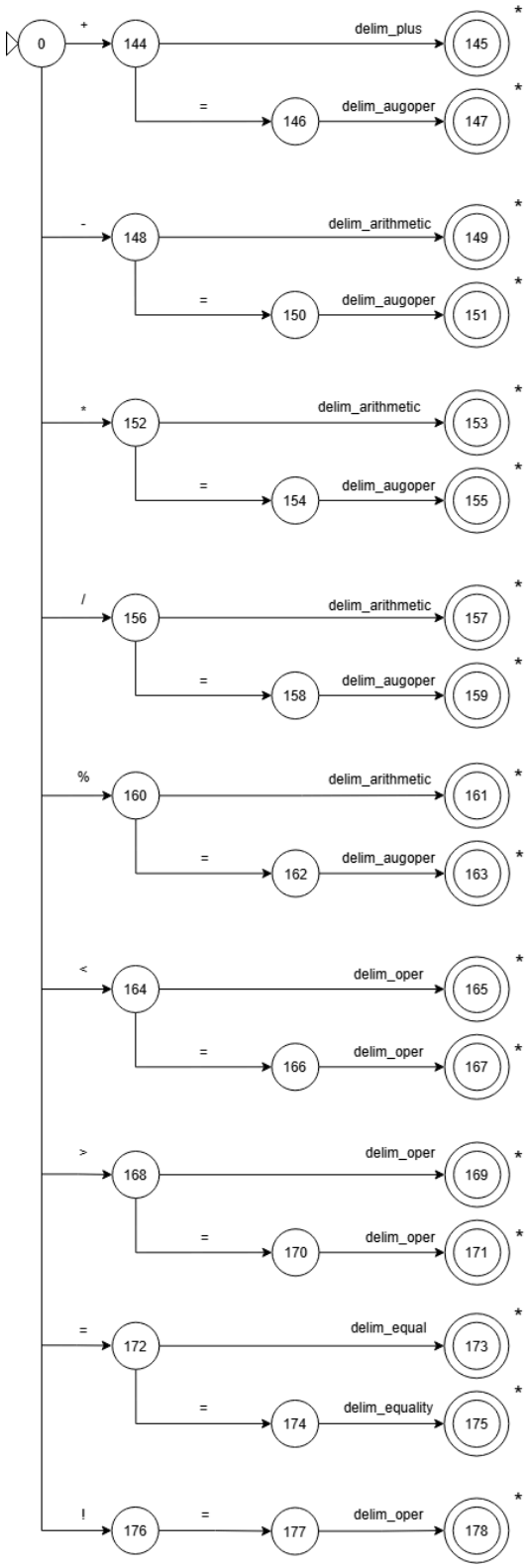
X. TRANSITION DIAGRAM

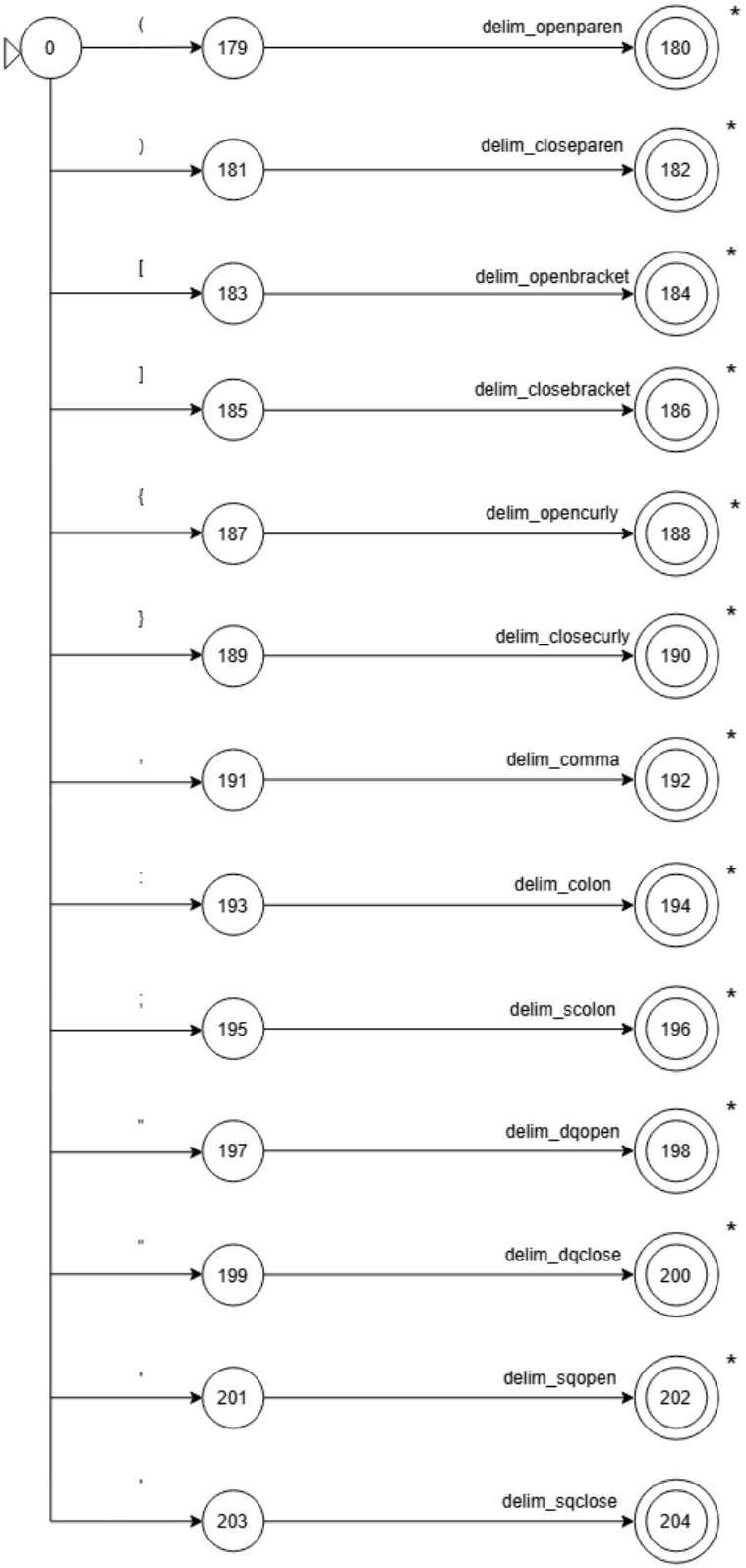
A. Reserved Words



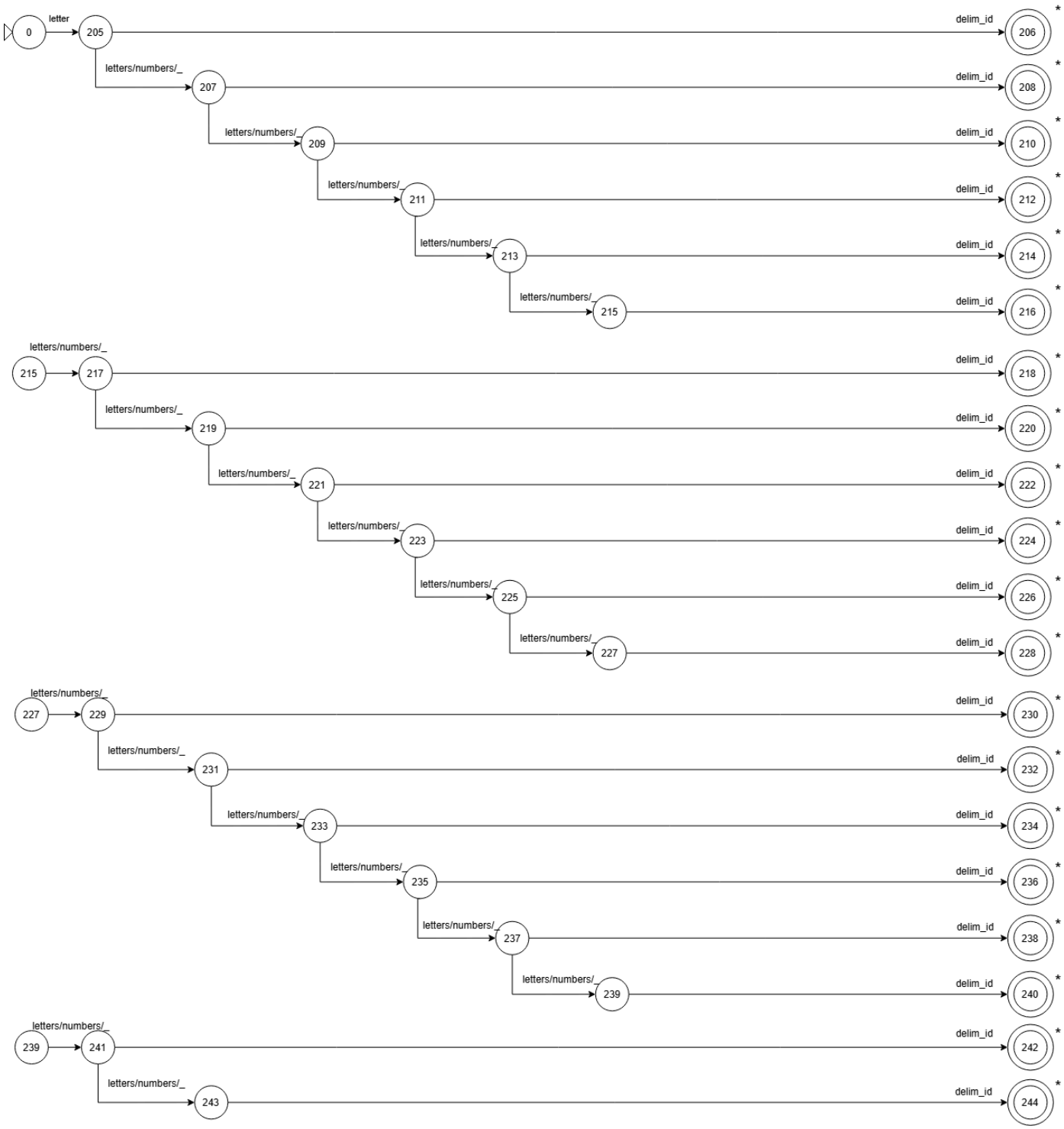

0
l
72
n
73
a
74
c
75
c
76
u
77
delim_bool
78
l
79
i
80
s
81
delim_datatype
82
o
83
o
84
d
85
delim_logic
86
n
87
e
88
v
89
e
90
delim_logic
91
o
92
t
93
h
94
a
95
n
96
delim_logic
97
p
98
a
99
s
100
delim_stmt
101
u
102
s
103
delim_stmt
104
e
105
r
106
s
107
i
108
s
109
delim_stmt
110
r
111
i
112
n
113
delim_io
114
r
115
e
116
o
117
r
118
delim_struc
119
s
120
t
121
r
122
i
123
n
124
a
125
delim_datatype
126
t
127
r
128
u
129
a
130
delim_datatype
131
T
132
r
133
u
134
a
135
delim_bool
136
w
137
h
138
a
139
r
140
f
141
a
142
delim_datatype
143

B. Reserved Symbols

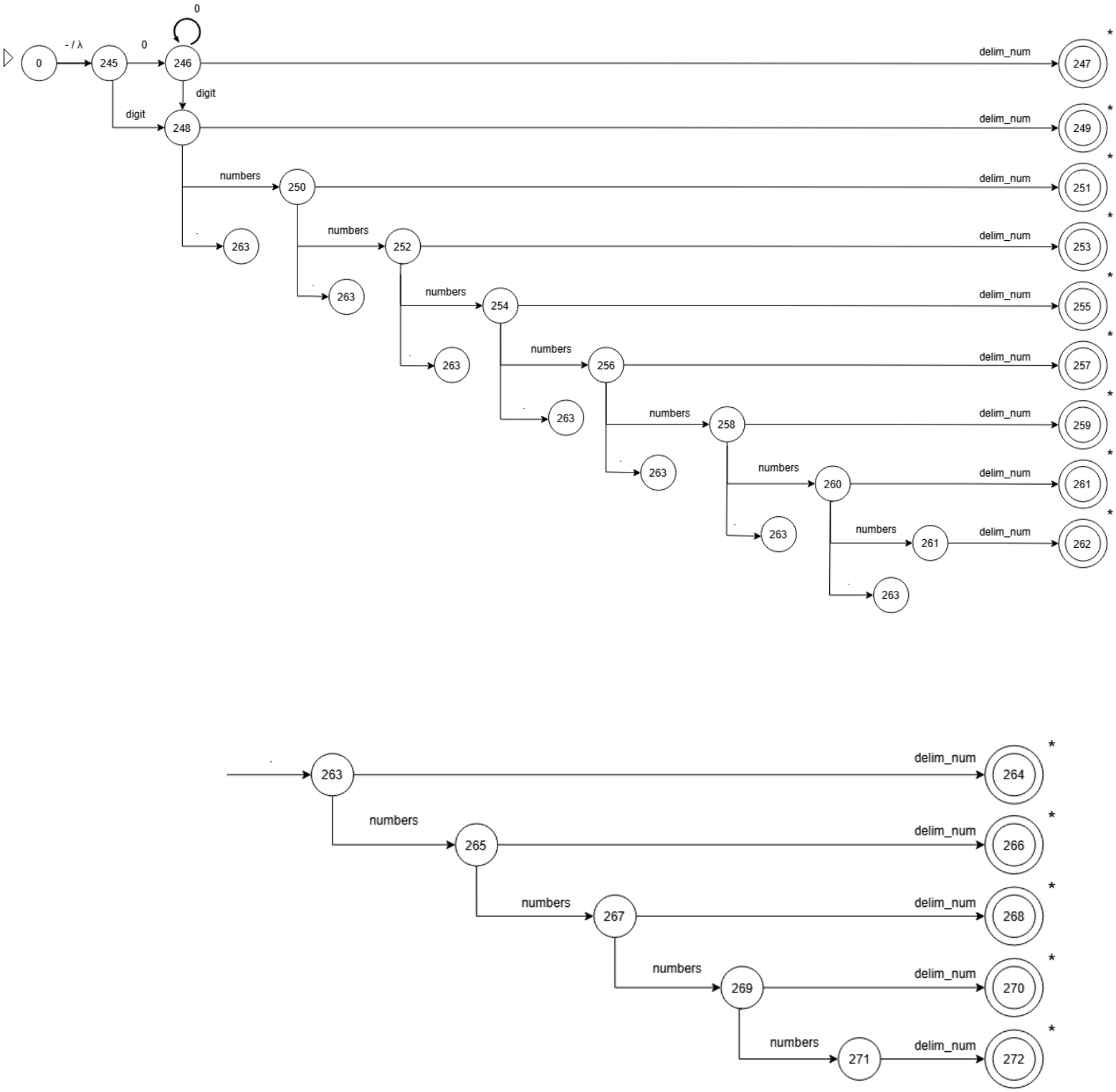



0
(
179
delim_openparen
180
*
)
181
delim_closeparen
182
*
[
183
delim_openbracket
184
*
]
185
delim_closebracket
186
*
{
187
delim_opencurly
188
*
}
189
delim_closecurly
190
*
,
191
delim_comma
192
*
:
193
delim_colon
194
*
;
195
delim_scolon
196
*
"
197
delim_dqopen
198
*
"
199
delim_dqclose
200
*
'
201
delim_sqopen
202
*
'
203
delim_sqclose
204
*

C. Identifiers



D. Integer and Float Literals



E. String Literals and Comments

